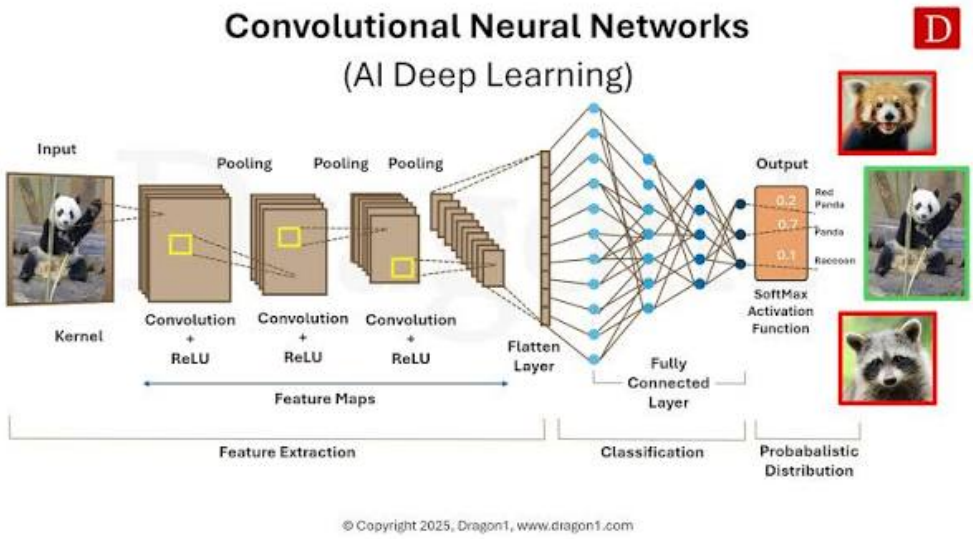
Session 3 Convolutional Neural Networks

Convolutional Neural Networks (CNN)



> **Convolutional Neural Network** adalah jenis jaringan saraf tiruan (*artificial neural network*) dalam bidang *deep learning* yang dirancang khusus untuk memproses data visual seperti gambar dan video

https://adamharley.com/nn_vis/cnn/3d.html

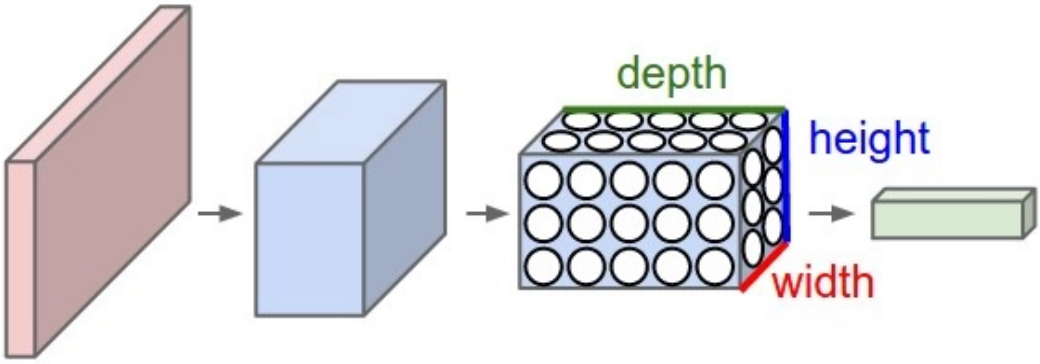
1. Mengapa CNN untuk gambar?

Gambar bukan sekadar vector panjang. Pixel memiliki **struktur spasial**: pixel yang berdekatan biasanya saling berhubungan. Fully Connected Network yang langsung menerima seluruh gambar mengabaikan struktur lokal ini dan cepat menghasilkan sangat banyak parameter.

CNN menggunakan dua ide utama:

- **Local connectivity** — neuron melihat patch lokal, bukan seluruh gambar sekaligus.
- **Weight sharing** — kernel/filter yang sama digunakan di banyak posisi gambar.

Akibatnya CNN dapat mempelajari pola visual dengan parameter yang jauh lebih efisien.



Perbedaan representasi fully connected network dan ConvNet: CNN mempertahankan dimensi spasial sebagai volume width × height × depth.

Visualisasi alur besar

```
IMAGE
  ↓
[ Convolution ] → feature map: edge / line / color
  ↓
[ ReLU ]
  ↓
[ Pooling ]
  ↓
[ Convolution ] → feature map: texture / shape
  ↓
[ ReLU + Pooling ]
  ↓
[ Flatten / Global Average Pooling ]
  ↓
[ Dense / Classifier ]
  ↓
CAT | DOG | ...
```

> **Intuisi:** layer awal belajar fitur sederhana seperti edge; layer lebih dalam mengombinasikannya menjadi texture, bagian objek, dan representasi yang semakin abstrak. Ini kecenderungan umum, bukan aturan mutlak.

2. Input image sebagai tensor

Untuk RGB, sebuah gambar biasanya direpresentasikan sebagai tensor:

$$\underbrace{H}_{\text{Height}} \times \underbrace{W}_{\text{Width}} \times \underbrace{C}_{\text{Channels}}$$

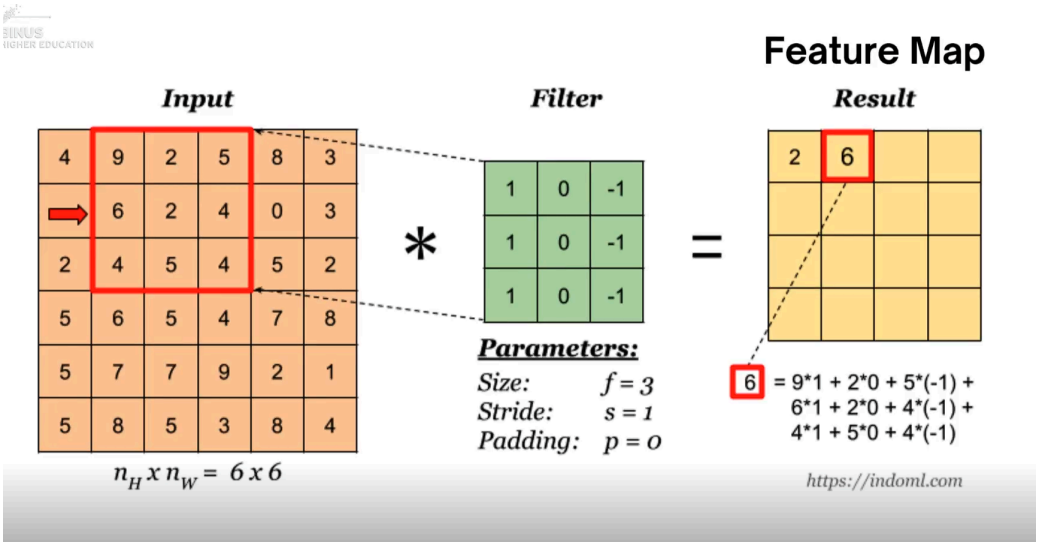
Contoh: gambar 32×32 RGB memiliki bentuk $32 \times 32 \times 3$, dengan channel **Red, Green, Blue**.

CNN mempertahankan struktur H dan W selama ekstraksi fitur, berbeda dari pendekatan yang langsung melakukan flatten di awal.

3. Convolution: inti CNN

Sebuah **kernel/filter** kecil, misalnya 3×3, digeser di atas gambar. Pada setiap posisi, elemen patch input dikalikan dengan bobot kernel lalu dijumlahkan (ditambah bias). Hasil di semua posisi membentuk **feature map**.

Visualisasi kernel sederhana



Kernel tidak harus kita tentukan manual. Saat training, CNN **mempelajari bobot kernel** melalui backpropagation.

Untuk input multi-channel, sebuah filter membentang sepanjang seluruh channel input. Banyak filter menghasilkan banyak output channel/feature maps.

Ukuran output convolution

Untuk satu dimensi spasial:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

dengan N = ukuran input, F = kernel size, P = padding, dan S = stride.

Contoh: input 32×32, kernel 3×3, stride 1, padding 1:

$$O = \frac{32 - 3 + 2(1)}{1} + 1 = 32$$

Maka spatial size tetap 32×32. Jika digunakan 32 filter, output menjadi $32 \times 32 \times 32$.

4. Stride, Padding, dan Feature Map

Stride menentukan seberapa jauh kernel berpindah. Stride 1 bergerak satu pixel; stride 2 sekaligus melakukan downsampling.

Padding menambahkan border, biasanya nol, di sekitar input. Padding `same` dengan stride 1 lazim dipakai agar ukuran spasial tidak cepat mengecil.

Feature map / activation map adalah respons sebuah filter pada berbagai lokasi. Satu filter dapat kuat merespons pola tertentu di banyak posisi karena bobotnya dibagikan secara spasial.

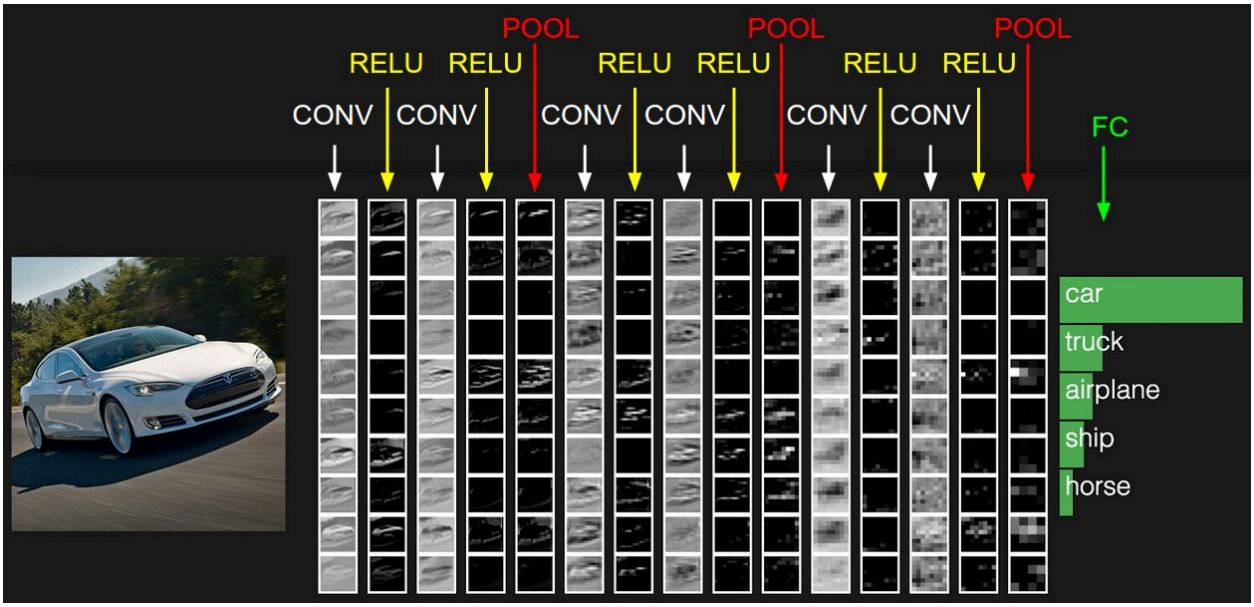
5. Activation Function — ReLU

Setelah convolution biasanya digunakan fungsi non-linear, misalnya ReLU:

$ReLU(x) = \max(0, x)$

```
x < 0 → 0
x > 0 → x
```

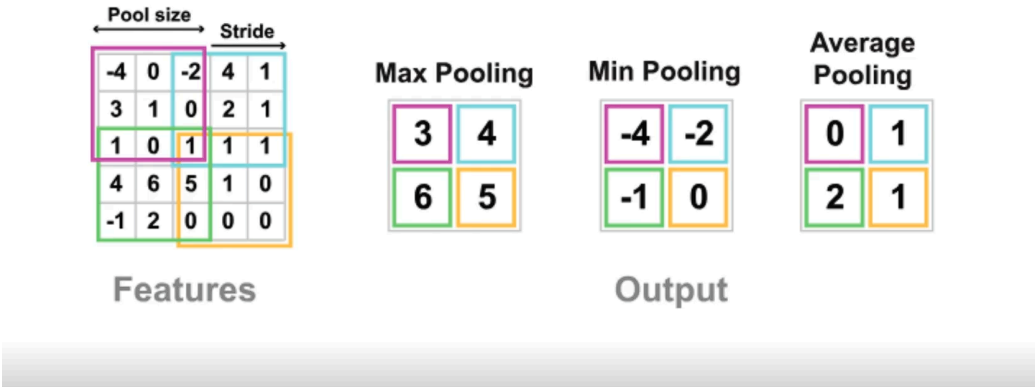
Tanpa non-linearity, tumpukan operasi linear tetap ekuivalen dengan satu transformasi linear dan tidak cukup untuk memodelkan pola visual kompleks.



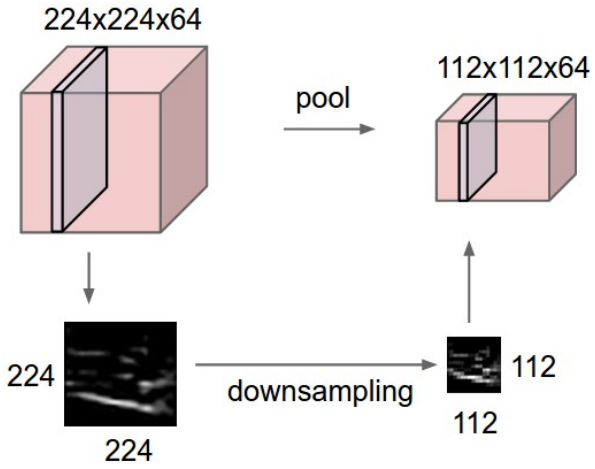
Contoh aliran aktivasi pada ConvNet: input secara bertahap diubah menjadi volume feature maps hingga menghasilkan class scores.

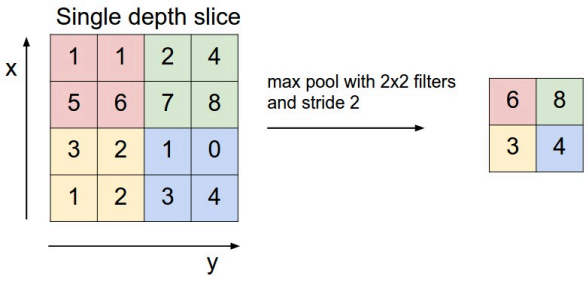
6. Pooling

Pooling melakukan **spatial downsampling** tanpa mempelajari bobot. Bentuk klasik adalah Max Pooling 2×2 stride 2.



Setiap blok 2×2 diganti nilai maksimum. Tinggi dan lebar menjadi setengah, sedangkan jumlah channel tetap.

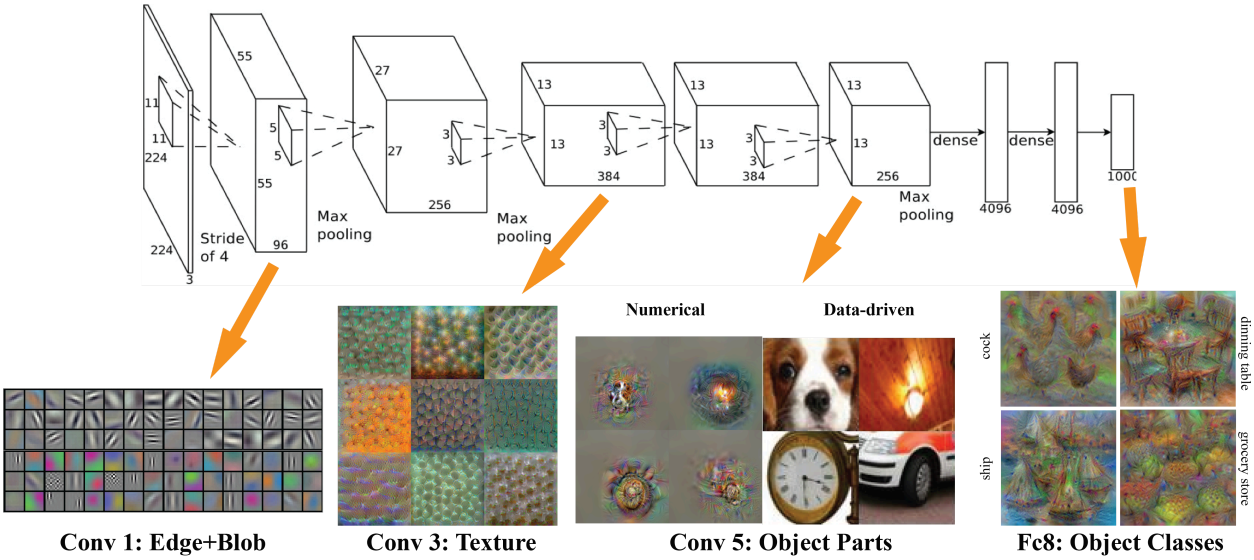




Max pooling memilih activation terbesar pada setiap area lokal.

Pooling mengecilkan width dan height tetapi mempertahankan depth/channel.

Manfaat praktis: mengurangi ukuran feature map dan komputasi. Arsitektur modern juga sering memakai **strided convolution** sebagai alternatif downsampling.



Visualisasi feature hierarchy pada CNN: layer awal menangkap edge/blob dan layer yang lebih dalam membentuk texture serta representasi yang lebih kompleks.

7. Receptive Field

Receptive field adalah area pada input asli yang dapat memengaruhi sebuah activation. Ketika layer ditumpuk, receptive field efektif membesar.

```
Layer 1: melihat pola lokal kecil
      ↓
Layer 2: menggabungkan beberapa pola Layer 1
```

```
      ↓
Layer 3+: melihat konteks yang lebih luas
```

Karena itu deep CNN dapat membangun hierarki representasi tanpa setiap neuron harus terhubung langsung ke semua pixel.

8. CNN vs Fully Connected Network

Aspek	Fully Connected	CNN
Koneksi	Semua neuron terhubung	Lokal
Struktur spasial	Cepat hilang setelah flatten	Dipertahankan
Weight sharing	Tidak	Ya
Parameter untuk gambar besar	Sangat banyak	Lebih efisien
Kegunaan utama	Data vector/tabular, classifier head	Data dengan struktur spasial seperti image

9. Apa itu Deep Convolutional Neural Network (DCNN)?

DCNN pada dasarnya adalah CNN dengan lebih banyak convolutional blocks/layers sehingga model dapat membangun representasi bertingkat yang lebih kompleks.

```
Input
 ↓
Conv → ReLU
 ↓        low-level features
Conv → ReLU
 ↓
Pool
 ↓
Conv → ReLU
 ↓        mid-level features
Conv → ReLU
 ↓
Pool
 ↓
Conv → ReLU
 ↓        higher-level representation
Global Pool / Dense
 ↓
Prediction
```

Menambah depth meningkatkan kapasitas dan receptive field, tetapi **lebih dalam tidak otomatis lebih baik**. Training dapat menjadi lebih sulit, kebutuhan compute meningkat, dan overfitting tetap mungkin terjadi.

10. Contoh arsitektur DCNN

Salah satu pola klasik:

```
INPUT
  ↓
CONV → ReLU
  ↓
CONV → ReLU
  ↓
MAX POOL
  ↓
CONV → ReLU
```

```
  ↓
CONV → ReLU
  ↓
MAX POOL
  ↓
GLOBAL AVG POOL / FLATTEN
  ↓
DENSE
  ↓
SOFTMAX
```

Pada layer awal, resolusi spasial relatif besar tetapi channel sedikit. Semakin dalam, resolusi biasanya mengecil sementara jumlah channel meningkat.

Membaca arsitektur tersebut

1. **Conv2D(32)**: belajar 32 jenis fitur lokal.
2. **MaxPooling2D**: mengurangi 32×32 menjadi kira-kira 16×16.
3. **Conv2D(64)**: membangun fitur yang lebih kaya dari feature maps sebelumnya.
4. **Pooling kedua**: mengurangi spatial resolution lagi.
5. **Conv2D(128)**: membentuk higher-level representation.
6. **GlobalAveragePooling2D**: merangkum setiap channel menjadi satu nilai tanpa flatten yang sangat besar.
7. **Dense(64)**: menggabungkan fitur untuk klasifikasi.
8. **Dropout(0.3)**: regularization saat training.
9. **Dense(10, softmax)**: menghasilkan distribusi probabilitas untuk 10 kelas.

> Jika definisi "3–5 layer" pada kelas hanya menghitung **trainable feature/classification layers**, contoh ini memiliki tiga convolution layers dan dua dense layers. Pooling, dropout, input, dan global pooling biasanya dijelaskan sebagai operasi/layer pendukung.

11. Visualisasi perubahan shape

Dengan model di atas:

```
32×32×3   Input
    ↓ Conv 32
32×32×32
    ↓ MaxPool
16×16×32
    ↓ Conv 64
16×16×64
    ↓ MaxPool
 8×8×64
    ↓ Conv 128
 8×8×128
    ↓ GlobalAveragePooling
128
    ↓ Dense
64
    ↓ Output
10 classes
```

Ini membantu mahasiswa melihat bahwa CNN perlahan mengubah **pixel space** → **feature representation** → **class probability**.

12. Training pipeline

```
Dataset
  ↓
Preprocessing / normalization
  ↓
Train / validation / test split
  ↓
Forward pass
  ↓
Prediction
  ↓
Loss
  ↓
Backpropagation
  ↓
Optimizer updates kernels/weights
  ↓
Repeat for many mini-batches
```

Untuk multiclass single-label classification, kombinasi umum adalah **softmax + cross-entropy**.

13. Evaluasi dan overfitting

Jangan hanya melihat training accuracy. Bandingkan training dan validation curves.

```
Healthy:
train loss ↓
val loss   ↓

Possible overfitting:
train loss ↓↓↓
val loss   ↓ lalu ↑
```

Selain accuracy, periksa confusion matrix dan performa per kelas jika distribusi kelas atau jenis kesalahan penting.

Cara mengurangi overfitting dapat mencakup:

- data augmentation,
- dropout,
- weight decay/regularization,
- early stopping,
- lebih banyak data,
- mengurangi kapasitas model bila terlalu besar.

14. Visualisasi feature maps

Feature map dapat divisualisasikan sebagai kumpulan gambar grayscale, satu untuk setiap filter. Jika convolution layer memiliki 32 filter, layer tersebut menghasilkan 32 activation maps. Filter yang berbeda akan memberikan respons kuat pada pola yang berbeda seperti edge, arah garis, texture, atau kombinasi fitur lainnya.

```
Original Image
      ↓
┌─────────┬─────────┬─────────┬─────────┐
```

```
| Map 1  | Map 2  | Map 3  |  ...   |
| edges  | lines  |texture |        |
└────────┴────────┴────────┴────────┘
      ↓ deeper convolution layers
┌────────┬────────┬────────┬────────┐
|Feature |Feature |Feature |  ...   |
|   A    |   B    |   C    |        |
└────────┴────────┴────────┴────────┘
```

Visualisasi ini menekankan bahwa CNN tidak menyimpan satu versi baru dari gambar, tetapi membentuk banyak channel representasi yang menyoroti fitur berbeda.

Gunakan demo ini untuk menunjukkan bahwa setiap filter dapat menghasilkan respons berbeda terhadap input yang sama.

15. Pengayaan — mengapa ResNet?

Ketika network menjadi sangat dalam, optimization menjadi lebih sulit. **ResNet (Residual Network)** memperkenalkan residual/skip connection agar sebuah block belajar residual $F(x)$ dan meneruskan input x secara langsung:

$y = F(x) + x$

Visualisasi residual block

```
        ┌──────────────────────┐
        │                      │
x ──────┼→ Conv → ReLU → Conv ─┼→ (+) → ReLU → y
        │                      ↑
        └──── shortcut / x ────┘
```

Shortcut menyediakan jalur langsung bagi informasi dan gradient. Paper ResNet menunjukkan residual learning membuat network yang jauh lebih dalam lebih mudah dioptimasi dibandingkan counterpart tanpa residual connection.

16. ResNet dan transfer learning

Untuk dataset yang relatif kecil, pendekatan praktis adalah memakai **ResNet yang telah dilatih pada ImageNet** sebagai feature extractor. Classification head asli dilepas dan diganti dengan classifier yang sesuai dengan jumlah kelas pada dataset baru.

Pada tahap awal, pretrained backbone biasanya dibekukan (**freeze**) sehingga hanya classification head baru yang dilatih. Setelah classifier stabil, sebagian layer backbone dapat dibuka kembali (**unfreeze**) dan dilatih dengan learning rate kecil. Tahap kedua ini disebut **fine-tuning**.

Transfer learning flow:

```
Image
  ↓
Pretrained ResNet50
  ↓
Learned ImageNet features
  ↓
Global Average Pooling
  ↓
New Dense classifier
```

```
  ↓
Your classes
```

Setelah head stabil, sebagian layer backbone dapat di-**unfreeze** dan dilatih dengan learning rate kecil (**fine-tuning**). Jangan langsung menganggap pretrained model selalu lebih baik; domain shift dan kualitas dataset tetap perlu dievaluasi.

17. CNN vs DCNN vs ResNet — ringkasan

Model	Inti	Cocok untuk menjelaskan
CNN sederhana	Conv + activation + downsampling + classifier	Fundamental image classification
DCNN	Lebih banyak convolutional blocks	Hierarki fitur yang lebih dalam
ResNet	DCNN + residual/skip connections	Training network yang sangat dalam

Referensi pengajar

- Stanford CS231n — *Convolutional Networks*: https://cs231n.github.io/convolutional-networks/
- He, Zhang, Ren, Sun — *Deep Residual Learning for Image Recognition*, CVPR 2016: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html
- TensorFlow/Keras — gunakan dokumentasi resmi untuk API `Conv2D`, `MaxPooling2D`, `GlobalAveragePooling2D`, dan `ResNet50`.

